JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Enum Class TimeUnit

java.lang.Object
    java.lang.Enum<TimeUnit>
        java.util.concurrent.TimeUnit

All Implemented Interfaces:

Serializable, Comparable<TimeUnit>, Constable

```
public enum TimeUnit
extends Enum<TimeUnit>
```

A `TimeUnit` represents time durations at a given unit of granularity and provides utility methods to convert across units, and to perform timing and delay operations in these units. A `TimeUnit` does not maintain time information, but only helps organize and use time representations that may be maintained separately across various contexts. A nanosecond is defined as one thousandth of a microsecond, a microsecond as one thousandth of a millisecond, a millisecond as one thousandth of a second, a minute as sixty seconds, an hour as sixty minutes, and a day as twenty four hours.

A `TimeUnit` is mainly used to inform time-based methods how a given timing parameter should be interpreted. For example, the following code will timeout in 50 milliseconds if the `lock` is not available:

```
Lock lock = ...;
if (lock.tryLock(50L, TimeUnit.MILLISECONDS)) ...
```

while this code will timeout in 50 seconds:

```
Lock lock = ...;
if (lock.tryLock(50L, TimeUnit.SECONDS)) ...
```

Note however, that there is no guarantee that a particular timeout implementation will be able to notice the passage of time at the same granularity as the given `TimeUnit`.

Since:

1.5

Nested Class Summary

Nested classes/interfaces declared in class Enum
Enum.EnumDesc<E>

Enum Constant Summary

Enum Constants

Enum Constant	Description
DAYS	Time unit representing twenty four hours.
HOURS	Time unit representing sixty minutes.
MICROSECONDS	Time unit representing one thousandth of a millisecond.
MILLISECONDS	Time unit representing one thousandth of a second.
MINUTES	Time unit representing sixty seconds.
NANOSECONDS	Time unit representing one thousandth of a microsecond.
SECONDS	Time unit representing one second.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`long`	**`convert`**`(long sourceDuration, TimeUnit sourceUnit)`	Converts the given time duration in the given unit to this unit.
`long`	**`convert`**`(Duration duration)`	Converts the given time duration to this unit.
`static TimeUnit`	**`of`**`(ChronoUnit chronoUnit)`	Converts a `ChronoUnit` to the equivalent `TimeUnit`.
`void`	**`sleep`**`(long timeout)`	Performs a `Thread.sleep` using this time unit.
`void`		

	timedJoin(Thread thread, long timeout)	Performs a timed Thread.join using this time unit.
void	**timedWait**(Object obj, long timeout)	Performs a timed Object.wait using this time unit.
ChronoUnit	**toChronoUnit**()	Converts this TimeUnit to the equivalent ChronoUnit.
long	**toDays**(long duration)	Equivalent to DAYS.convert(duration, this).
long	**toHours**(long duration)	Equivalent to HOURS.convert(duration, this).
long	**toMicros**(long duration)	Equivalent to MICROSECONDS.convert(duration, this).
long	**toMillis**(long duration)	Equivalent to MILLISECONDS.convert(duration, this).
long	**toMinutes**(long duration)	Equivalent to MINUTES.convert(duration, this).
long	**toNanos**(long duration)	Equivalent to NANOSECONDS.convert(duration, this).
long	**toSeconds**(long duration)	Equivalent to SECONDS.convert(duration, this).
static TimeUnit	**valueOf**(String name)	Returns the enum constant of this class with the specified name.
static TimeUnit[]	**values**()	Returns an array containing the constants

	of this enum class, in the order they are declared.

Methods declared in class Enum

`clone`, `compareTo`, `describeConstable`, `equals`, `finalize`, `getDeclaringClass`, `hashCode`, `name`, `ordinal`, `toString`, `valueOf`

Methods declared in class Object

`getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Enum Constant Details

NANOSECONDS

```
public static final TimeUnit NANOSECONDS
```

Time unit representing one thousandth of a microsecond.

MICROSECONDS

```
public static final TimeUnit MICROSECONDS
```

Time unit representing one thousandth of a millisecond.

MILLISECONDS

```
public static final TimeUnit MILLISECONDS
```

Time unit representing one thousandth of a second.

SECONDS

```
public static final TimeUnit SECONDS
```

Time unit representing one second.

MINUTES

```
public static final TimeUnit MINUTES
```

Time unit representing sixty seconds.

Since:

1.6

HOURS

```
public static final TimeUnit HOURS
```

Time unit representing sixty minutes.

Since:

1.6

DAYS

```
public static final TimeUnit DAYS
```

Time unit representing twenty four hours.

Since:

1.6

Method Details

values

```
public static TimeUnit[] values()
```

Returns an array containing the constants of this enum class, in the order they are declared.

Returns:

an array containing the constants of this enum class, in the order they are declared

valueOf

```
public static TimeUnit valueOf(String name)
```

Returns the enum constant of this class with the specified name. The string must match *exactly* an identifier used to declare an enum constant in this class. (Extraneous whitespace characters are not permitted.)

Parameters:

`name` - the name of the enum constant to be returned.

Returns:

the enum constant with the specified name

Throws:

`IllegalArgumentException` - if this enum class has no constant with the specified name

NullPointerException - if the argument is null

convert

```
public long convert(long sourceDuration,
                    TimeUnit sourceUnit)
```

Converts the given time duration in the given unit to this unit. Conversions from finer to coarser granularities truncate, so lose precision. For example, converting 999 milliseconds to seconds results in 0. Conversions from coarser to finer granularities with arguments that would numerically overflow saturate to Long.MIN_VALUE if negative or Long.MAX_VALUE if positive.

For example, to convert 10 minutes to milliseconds, use: TimeUnit.MILLISECONDS.convert(10L, TimeUnit.MINUTES)

Parameters:

sourceDuration - the time duration in the given sourceUnit

sourceUnit - the unit of the sourceDuration argument

Returns:

the converted duration in this unit, or Long.MIN_VALUE if conversion would negatively overflow, or Long.MAX_VALUE if it would positively overflow.

convert

```
public long convert(Duration duration)
```

Converts the given time duration to this unit.

For any TimeUnit unit, unit.convert(Duration.ofNanos(n)) is equivalent to unit.convert(n, NANOSECONDS), and unit.convert(Duration.of(n, unit.toChronoUnit())) is equivalent to n (in the absence of overflow).

API Note:

This method differs from Duration.toNanos() in that it does not throw ArithmeticException on numeric overflow.

Parameters:

duration - the time duration

Returns:

the converted duration in this unit, or Long.MIN_VALUE if conversion would negatively overflow, or Long.MAX_VALUE if it would positively overflow.

Throws:

NullPointerException - if duration is null

Since:

11

See Also:

`Duration.of(long,TemporalUnit)`

toNanos

```
public long toNanos(long duration)
```

Equivalent to `NANOSECONDS.convert(duration, this)`.

Parameters:

`duration` - the duration

Returns:

the converted duration, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

toMicros

```
public long toMicros(long duration)
```

Equivalent to `MICROSECONDS.convert(duration, this)`.

Parameters:

`duration` - the duration

Returns:

the converted duration, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

toMillis

```
public long toMillis(long duration)
```

Equivalent to `MILLISECONDS.convert(duration, this)`.

Parameters:

`duration` - the duration

Returns:

the converted duration, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

toSeconds

```
public long toSeconds(long duration)
```

Equivalent to `SECONDS.convert(duration, this)`.

Parameters:

duration - the duration

Returns:

the converted duration, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

toMinutes

```
public long toMinutes(long duration)
```

Equivalent to `MINUTES.convert(duration, this)`.

Parameters:

duration - the duration

Returns:

the converted duration, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

Since:

1.6

toHours

```
public long toHours(long duration)
```

Equivalent to `HOURS.convert(duration, this)`.

Parameters:

duration - the duration

Returns:

the converted duration, or `Long.MIN_VALUE` if conversion would negatively overflow, or `Long.MAX_VALUE` if it would positively overflow.

Since:

1.6

toDays

```
public long toDays(long duration)
```

Equivalent to `DAYS.convert(duration, this)`.

Parameters:

duration - the duration

Returns:

the converted duration

Since:

1.6

timedWait

```
public void timedWait(Object obj,
                      long timeout)
               throws InterruptedException
```

Performs a timed `Object.wait` using this time unit. This is a convenience method that converts timeout arguments into the form required by the `Object.wait` method.

For example, you could implement a blocking `poll` method (see `BlockingQueue.poll`) using:

```
public E poll(long timeout, TimeUnit unit)
    throws InterruptedException {
  synchronized (lock) {
    while (isEmpty()) {
      unit.timedWait(lock, timeout);
      ...
    }
  }
}
```

Parameters:

`obj` - the object to wait on

`timeout` - the maximum time to wait. If less than or equal to zero, do not wait at all.

Throws:

`IllegalMonitorStateException` - if the current thread is not the owner of the object's monitor.

`InterruptedException` - if interrupted while waiting

timedJoin

```
public void timedJoin(Thread thread,
                      long timeout)
               throws InterruptedException
```

Performs a timed `Thread.join` using this time unit. This is a convenience method that converts time arguments into the form required by the `Thread.join` method.

Parameters:

`thread` - the thread to wait for

`timeout` - the maximum time to wait. If less than or equal to zero, do not wait at all.

Throws:

`InterruptedException` - if interrupted while waiting

sleep

```
public void sleep(long timeout)
           throws InterruptedException
```

Performs a `Thread.sleep` using this time unit. This is a convenience method that converts time arguments into the form required by the `Thread.sleep` method.

Parameters:

`timeout` - the minimum time to sleep. If less than or equal to zero, do not sleep at all.

Throws:

`InterruptedException` - if interrupted while sleeping

toChronoUnit

```
public ChronoUnit toChronoUnit()
```

Converts this `TimeUnit` to the equivalent `ChronoUnit`.

Returns:

the converted equivalent ChronoUnit

Since:

9

of

```
public static TimeUnit of(ChronoUnit chronoUnit)
```

Converts a `ChronoUnit` to the equivalent `TimeUnit`.

Parameters:

`chronoUnit` - the ChronoUnit to convert

Returns:

the converted equivalent TimeUnit

Throws:

`IllegalArgumentException` - if `chronoUnit` has no equivalent TimeUnit

NullPointerException - if chronoUnit is null

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.